

Rescue Robot

Team Guide

RoboCup Junior Australia - Open Rescue Line 2026
Melbourne High School

What is in this guide?

1. How our robot works (the simple version)
2. Setting up your laptop, then step by step: put code on the robot and run it
3. The important files and what each one does
4. The numbers you can change to make the robot behave better
5. Test scripts to check each part works
6. The problems we still have right now
7. What to do when you see an error
8. How to ask AI for help

GOOD TO KNOW: You do NOT need to understand all the code. You only need to know which file to run, how to copy it to the robot, and which number to change. This guide shows you all three. Everything here is done from **your own laptop**.

1. How our robot works

Think of the robot as having **eyes**, a **brain** and **muscles**:

Part	What it is	What it does
Brain	Raspberry Pi (name: <code>rescue.local</code>)	Runs our Python code. We log in to it from a laptop.
Line eyes	8 IR sensors in a row under the front (Parallax #28034)	Each sensor says black (1) or white (0). Together they tell us where the black line is.
Camera	Pi camera on a servo arm	Sees green squares, the silver tape, balls and the coloured triangles.
Distance sensor	ToF sensor, 19 mm off the floor, at the front	Measures how far away something is in front (in mm).
Tilt sensor	IMU (BNO085)	Tells us when we are going up a ramp so we can speed up.
Muscles	2 front wheel motors (rear right motor is broken)	Drive and turn the robot.
Claw	On the BACK of the robot. 2 servos: open/close + lift	Picks up balls in the evacuation zone.

How line following works, in one sentence: 80 times every second, the robot reads the 8 sensors, works out how far the line is from the middle, and steers back toward it. The further off the line is, the harder it steers. This is called **PID**.

The error number: the code turns the 8 sensors into one number called the **error**. 0 means the line is right in the middle. +3.5 means it is all the way at one end (OUT0), -3.5 means all the way at the other end (OUT7).

What it does at special places:

Situation	What the robot does
Line lost (all 8 sensors white)	Spins on the spot toward the last way it was turning (LEFT or RIGHT) until it sees the line again.
Green square	Stops for 1 s, checks again which side the green is on (votes), drives forward a little, then turns that way.
Sharp 90 degree corner after a straight	Drives forward a little so the wheels reach the corner, then pivots.
U-turn / two lines seen at once	Follows only the line closest to where it was before. Slows down in sharp bends.
Silver tape (evacuation zone)	Only in <code>pid_line_follow.py</code> : stops, checks, then runs the endzone code.
Obstacle (bottle)	Currently TURNED OFF (<code>USE_TOF = False</code>).

2. Set up YOUR laptop (do this only once)

Everyone uses their **own laptop**. The **Pi on the robot** holds the team's real, up-to-date copy of the code. Your laptop gets a copy from the Pi, you change it, and you send it back.

GOOD TO KNOW: Think of the Pi as the team's shared notebook. Always read the newest page from it before you write, and put your page back when you are done.

Setup 1 - Check your laptop can talk to the Pi

Windows: press the Windows key, type **PowerShell**, press Enter. **Mac:** open **Terminal** (Cmd + Space, type Terminal).

Type this (robot must be switched on, same Wi-Fi):

```
ssh nimesh@rescue.local
```

The first time it asks *"Are you sure you want to continue connecting?"* - type **yes** and press Enter. Then type the Pi password (ask the team). If you see `nimesh@rescue:~ $`, it works. Type `exit` to go back to your laptop.

Setup 2 - Download the code from the Pi to your laptop

Windows PowerShell (on your laptop):

```
scp -r nimesh@rescue.local:~/rescue_robot "$HOME\Documents\"
```

Mac Terminal (on your laptop):

```
scp -r nimesh@rescue.local:~/rescue_robot ~/Documents/
```

This copies the whole `rescue_robot` folder into your **Documents** folder. `-r` means "copy the folder and everything inside it".

Setup 3 - Install a code editor (recommended)

Download **Visual Studio Code** (free) from code.visualstudio.com. Open it, then File -> Open Folder -> Documents -> `rescue_robot`. You can now click on any file to read or change it. Notepad also works if you can't install anything.

Your folder paths (use these everywhere in this guide)

Where	Folder
Your laptop (Windows)	<code>\$HOME\Documents\rescue_robot\line_follow</code> (<code>\$HOME</code> = your user folder)
Your laptop (Mac)	<code>~/Documents/rescue_robot/line_follow</code>
The Pi	<code>~/rescue_robot/line_follow</code> (~ means <code>/home/nimesh</code>)
Pi login	<code>nimesh@rescue.local</code>

CAREFUL: Only **ONE** person sends files to the Pi at a time. If two people change the same file on their own laptops and both copy it to the Pi, the second one wipes out the first one's change. Say in the team chat "I'm pushing `pid_line_follow_no_endzone.py`" before you copy.

2. (continued) Step by step: every time you work on the robot

The code lives in **two places**: on **your laptop** (where you edit it) and on the **Pi** (where it runs). When you change a file on your laptop, you must **copy it to the Pi** or the robot will still run the old version.

GOOD TO KNOW: Golden rule: get the latest from the Pi -> edit on your laptop -> copy back with scp -> run on the Pi.

Step 1 - Turn on the robot and connect to the same Wi-Fi

Turn the robot on and wait about 1 minute for the Pi to start. Your laptop must be on the same network as the Pi.

Step 2 - Open PowerShell (Windows) or Terminal (Mac) on your laptop

A window opens where you type commands. Everything in a grey box marked **on your laptop** goes here.

Step 3 - Get the latest code from the Pi

Windows PowerShell (on your laptop):

```
scp -r nimesh@rescue.local:~/rescue_robot "$HOME\Documents\"
```

Mac Terminal (on your laptop):

```
scp -r nimesh@rescue.local:~/rescue_robot ~/Documents/
```

CAREFUL: This REPLACES the copy on your laptop with the Pi's copy. If you changed something on your laptop and haven't sent it to the Pi yet, send it first (Step 5) or it will be lost.

Step 4 - Go into the code folder and make your change

Windows PowerShell (on your laptop):

```
cd "$HOME\Documents\rescue_robot\line_follow"
```

Mac Terminal (on your laptop):

```
cd ~/Documents/rescue_robot/line_follow
```

cd means "change directory" = go into that folder. Now open the file in VS Code, change the number, and **save** (Ctrl + S, or Cmd + S on Mac).

Step 5 - Copy the file(s) to the Pi with scp

Copy one file (on your laptop, inside the line_follow folder):

```
scp pid_line_follow_no_endzone.py nimesh@rescue.local:~/rescue_robot/line_follow/
```

Copy ALL the main files at once (on your laptop - safest after big changes):

```
scp camera_servo.py camera_watcher.py zone_vision.py endzone.py claw.py test_endzone.py  
aim_camera.py bottle_detector.py pid_line_follow_no_endzone.py pid_line_follow.py  
pid_line_follow_only.py nimesh@rescue.local:~/rescue_robot/line_follow/
```

It will ask for the Pi's password. Type it (you will NOT see the letters appear - that is normal) and press Enter. Each file prints **100%** when it worked.

CAREFUL: If a file says **Permission denied**, it did NOT copy. See section 7 for the fix.

Step 6 - Log in to the Pi with ssh

On your laptop:

```
ssh niges@rescue.local
```

Type the password. When it works, the start of the line changes to something like `niges@rescue:~`. Now everything you type runs **on the robot**.

Step 7 - Go to the project folder on the Pi

On the Pi (after ssh):

```
cd ~/rescue_robot
```

Step 8 - Set the camera angle by hand

Gently move the camera arm by hand to the angle that works for line following (the one we found best). The line-following code does NOT move the camera any more, so it stays where you put it.

Step 9 - Put the robot on the line and run the code

On the Pi - the main file to use for now:

```
python3 line_follow/pid_line_follow_no_endzone.py
```

Check the first lines it prints. You should see something like:

```
[version: lost-line -> spin on the spot toward last LEFT/RIGHT turn, LOOP_HZ=80]
```

If you do NOT see this line, the Pi has an OLD copy. Go back to Step 5.

Step 10 - Stop the robot

Press **Ctrl + C** in the window. The motors stop. Pick the robot up if it is about to fall off.

Step 11 - Log out of the Pi

On the Pi:

```
exit
```

You are now back on your laptop.

Other useful things to do

Change a number (tune the robot)

1. Open the file on the laptop (e.g. in VS Code or Notepad).
2. Find the number near the TOP of the file (all the settings are at the top, in CAPITAL LETTERS).
3. Change it and **save** (Ctrl + S).
4. Copy it to the Pi again (Step 5).
5. Run it again (Step 9).

TIP: Only change ONE number at a time, and write down what you changed and what happened. Otherwise you will not know which change helped.

Look at what the camera sees

On the Pi:

```
python3 line_follow/aim_camera.py
```

Type a servo number (like 0.3) and press Enter. It moves the camera, holds it there, and saves a picture. Then on your laptop (open a second PowerShell/Terminal window, go to your rescue_robot folder):

Windows (on your laptop):

```
scp niges@rescue.local:~/cam_check.jpg . ; start cam_check.jpg
```

Mac (on your laptop):

```
scp niges@rescue.local:~/cam_check.jpg . ; open cam_check.jpg
```

This copies the picture to the laptop and opens it. Type **q** on the Pi to quit aim_camera.

Edit a file directly on the Pi (quick fix only)

On the Pi:

```
nano line_follow/pid_line_follow_no_endzone.py
```

Use the arrow keys to move. **Ctrl + O** then Enter = save. **Ctrl + X** = quit.

CAREFUL: If you edit on the Pi, your laptop copy is now old. Before you change anything on your laptop again, get the latest from the Pi first (Step 3), or your next copy will undo the Pi change.

Where the files are

Place	Path
Your laptop (Windows)	\$HOME\Documents\rescue_robot\line_follow
Your laptop (Mac)	~/Documents/rescue_robot/line_follow
Pi	~/rescue_robot/line_follow (~ means /home/niges)
Pi login	niges@rescue.local

3. The important files

All of these are in the `line_follow` folder unless it says otherwise.

Files you RUN

File	What it does	Use it when...
<code>pid_line_follow_no_endzone.py</code>	Main program right now. Line following + green squares + corners + U-turns. Ignores silver tape and has NO endzone.	Testing the course. Use this one most of the time.
<code>pid_line_follow.py</code>	Same as above PLUS silver tape detection and the evacuation zone.	Full runs - but NOT yet (see Problems: silver detector).
<code>pid_line_follow_only.py</code>	ONLY line following. No camera, no green, no obstacles.	Tuning Kp / Kd / speed on plain line tiles.
<code>test_endzone.py</code>	Runs the evacuation zone code by itself. Put the robot at the zone entrance.	Testing balls + claw + triangles. Add a number for how many balls: <code>test_endzone.py 3</code>
<code>zone_vision.py</code>	Camera-only test in the zone. Prints balls/triangles it sees, saves <code>~/ez_debug.jpg</code> .	Checking the camera can see balls/triangles BEFORE driving.
<code>camera_watcher.py</code>	Camera-only test. Prints green side + silver, saves <code>~/cam_check.jpg</code> .	Checking green-square detection.
<code>aim_camera.py</code>	Move the camera servo to a number you type and save a picture.	Finding good camera angles.
<code>../sensor_tests/test_line8.py</code>	Shows the 8 line sensors live, like <code>..##....</code>	Checking the line sensor wires work.

Helper files (other files use these - you rarely run them yourself)

File	What is inside
<code>camera_watcher.py</code>	Runs the camera in the background. Finds green squares and which side of the black line they are on. Also flips the picture (the camera is mounted upside down).
<code>camera_servo.py</code>	Moves the camera arm. Has the angle numbers <code>LINE_VALUE</code> and <code>LOOK_UP_VALUE</code> .
<code>endzone.py</code>	The evacuation zone plan: find ball -> drive to it -> turn 180 -> grab -> find triangle -> drop.
<code>zone_vision.py</code>	Finds balls (silver = alive, black = dead) and the red/green triangles in a picture.
<code>claw.py</code>	Open, close, lower and lift the claw.
<code>../control/motors.py</code>	Talks to the motor drivers. <code>set_speeds(left, right)</code> , -100 to 100.
<code>tof_detector.py</code> / <code>tof_sensor.py</code>	Reads the distance sensor.
<code>bottle_detector.py</code> / <code>obstacle_evade.py</code>	Checks if an obstacle is a bottle, and drives around it.
<code>accel_speed_boost.py</code> / <code>imu_sensor.py</code>	Speeds up on ramps using the tilt sensor.
<code>spill_classifier.py</code> / <code>spill_model_weights.py</code>	The trained 'is this silver tape?' detector.

GOOD TO KNOW: Files ending in `.bak . . .` are backups of older versions. If a change made things worse, you can copy the backup back. Do not delete them yet.

4. The numbers you can change (tuning)

All of these are near the top of `pid_line_follow_no_endzone.py` (and the same names are in `pid_line_follow.py` and `pid_line_follow_only.py` - change all of them so they match).

Line following

Name	Now	What it means	If the robot... then...
BASE_SPEED	30	Normal forward speed (0-100).	goes off on bends -> lower it. Too slow -> raise it.
Kp	15	How hard it steers for how far off the line is.	is late into corners -> raise. Zig-zags on straights -> lower.
Kd	15	Extra kick when the line moves suddenly; also calms wobbles.	wobbles after turns -> raise. Is jerky -> lower.
LOOP_HZ	80	How many times per second it reads the sensors.	Leave it.
JUNCTION_MIN_BLACK	6	This many sensors black in a row = a junction, drive straight.	Leave it.

When the line is lost

Name	Now	What it means	If the robot... then...
LOST_TURN_SPEED	25	How fast it spins on the spot to find the line.	overshoots the line -> lower. Too slow -> raise.
TURN_DIR_MIN_CORRECTION	10	How big a steer counts as a real LEFT or RIGHT turn.	remembers the wrong direction after a wobble -> raise to 15-20.

Slowing down in sharp bends (U-turns)

Name	Now	What it means	If the robot... then...
MIN_SPEED_FRAC	0.35	Slowest speed in sharp bends, as a part of BASE_SPEED ($0.35 \times 30 = \text{about } 10$).	still runs off U-turns -> lower to 0.25. Stalls in bends -> raise.
SLOWDOWN_START_ERR	1.0	Starts slowing down when the line is this far off centre.	slows too often -> raise to 1.5.
SLOWDOWN_FULL_ERR	3.0	Slowest when the line is this far off centre.	Usually leave it.

Sharp corners

Name	Now	What it means	If the robot... then...
CORNER_ENABLED	True	Turn the corner routine on/off.	does strange things at corners -> set False.
CORNER_FORWARD_S	0.25	Seconds to drive forward before pivoting at a corner.	U-turns at a corner -> raise. Overshoots -> lower.
CORNER_MIN_STRAIGHT_S	0.4	Must have been going straight this long first.	corner routine never prints 'corner' -> lower.

Green squares

Name	Now	What it means	If the robot... then...
GREEN_STOP_S	1.0	Stops this long after seeing green (and votes left/right).	Leave it.
GREEN_FORWARD_S	0.4	Drives forward this long before turning.	turns too early -> raise (0.5, 0.6...). Too late -> lower.

Name	Now	What it means	If the robot... then...
GREEN_TURN_DURATION_S	0.6	How long the green turn lasts.	under-turns -> raise. Over-turns -> lower.
GREEN_TURN_SPEED	30	How fast it turns at green.	Usually leave it.
GREEN_COOLDOWN_S	0.8	Ignores green for this long after a green turn.	misses a 2nd square right after -> lower.
GREEN_SWAP_TURN	False	Swap the turn direction.	says 'turning left' but turns right -> set True.

Camera and obstacles

Name (file)	Now	What it means
SET_CAMERA_ANGLE (pid files)	False	False = the code does not move the camera at the start. You set it by hand.
LINE_VALUE (camera_servo.py)	0.30	Camera angle the code goes BACK to after an obstacle check / the endzone. Should match your hand angle (find it with aim_camera.py).
LOOK_UP_VALUE (camera_servo.py)	1.0	Camera angle for looking ahead (endzone + obstacles). 1.0 is the highest the servo can go.
MIRROR_LEFT_RIGHT / FLIP_UP_DOWN (camera_watcher.py)	True / True	Turns the camera picture the right way up (the camera is mounted upside down). Do not change.
USE_TOF (pid files)	False	Obstacle checking is OFF. Set True to turn it back on.
OBSTACLE_TRIGGER_MM (pid files)	50	Distance (mm) that counts as 'something in front'.

Evacuation zone (endzone.py and claw.py)

Name	Now	What it means	If the robot... then...
TURN_180_S	1.2	Seconds to spin for a 180 degree turn (it is timed).	turns too far / not enough -> lower / raise.
BALL_STOP_MM	60	Stops this far (ToF) from a ball, so the claw lands on it after the 180.	claw misses short -> lower. Hits the ball -> raise.
DROP_STOP_MM	= BALL_STOP_MM	Same, for the triangles.	Change BALL_STOP_MM.
DROP_LOWER_FRACTION	0.5	How far the claw goes down when dropping (half).	ball doesn't fall in -> raise.
MAX_BALLS	3	How many balls to rescue.	Leave it.
TIME_BUDGET_S	200	Gives up on the zone after this many seconds.	Leave it.
LIFT_DOWN_TIME_S / LIFT_UP_TIME_S (claw.py)	1.5 / 0.5	How long the lift servo spins down / up.	claw doesn't reach the floor -> raise down time.

5. Test in this order

If something does not work, go back to the earliest test that fails. Run all of these **on the Pi** from the `~/rescue_robot` folder.

#	Command (on the Pi)	What you should see
1	<code>python3 sensor_tests/test_line8.py</code>	Slide black tape under the sensors: the # moves smoothly from one end to the other.
2	<code>python3 line_follow/pid_line_follow_only.py</code>	Robot follows a plain line. Tune Kp/Kd here.
3	<code>python3 line_follow/camera_watcher.py</code>	Hold a green square on each side of the line: it prints the correct side.
4	<code>python3 line_follow/pid_line_follow_no_endzone.py</code>	Full course without the zone. Watch the printed messages.
5	<code>python3 line_follow/zone_vision.py</code>	In the zone: it lists balls and triangles. Copy <code>~/ez_debug.jpg</code> to the laptop and check.
6	<code>python3 line_follow/test_endzone.py</code>	Rescues ONE ball.
7	<code>python3 line_follow/test_endzone.py 3</code>	Rescues all three.

Messages the robot prints (and what they mean)

Message	Meaning
<code>[version: ... LOOP_HZ=80]</code>	The new code is running. If missing, the Pi has an old copy.
<code>TURN: LEFT / TURN: RIGHT / TURN: STRAIGHT</code>	What the robot is doing right now (only printed when it changes).
<code>LINE LOST -> ... spinning LEFT on the spot ...</code>	It lost the line and is spinning toward its last real turn.
<code>LINE FOUND again</code>	Back on the line.
<code>GREEN seen on the right (green_x=., line_x=.)</code>	It saw a green square. <code>green_x/line_x</code> = where in the picture.
<code>GREEN confirmed: right (votes left=2 right=19)</code>	After the 1 s stop, the vote result. If the votes are close, the camera is unsure.
<code>corner -> + side</code>	The sharp-corner routine started.

6. Problems we still have right now

Be honest in testing: if one of these happens, it is a KNOWN problem, not your fault.

#	Problem	What it causes	What to do for now
1	Silver detector fires on green tiles.	In <code>pid_line_follow.py</code> the robot could think a green square is the evacuation zone and start the endzone in the middle of the course.	Use <code>pid_line_follow_no_endzone.py</code> . The silver classifier needs re-training with pictures of green tiles.
2	Green side detection is not 100%.	Sometimes turns the wrong way at green squares.	The 1 s vote helps. Send the printed votes + <code>green_x/line_x</code> to whoever is fixing it.
3	Line-lost behaviour still being tuned.	Sometimes spins the wrong way or too far after losing the line.	Watch the 'LINE LOST' message: does it say the right direction? Tune <code>LOST_TURN_SPEED</code> .
4	U-turn tiles and sharp corners.	Can run off the end of a hairpin or U-turn at a corner.	Tune <code>MIN_SPEED_FRAC</code> and <code>CORNER_FORWARD_S</code> .
5	Obstacle avoidance is OFF.	The robot will drive into the bottle.	<code>USE_TOF = False</code> for now. Needs testing before turning on.
6	Evacuation zone not tested on the real robot yet.	Unknown - balls, claw position and 180 turns all need tuning.	Test with <code>zone_vision.py</code> first, then <code>test_endzone.py</code> .
7	Leaving the evacuation zone is not written yet.	After the balls, the run just ends.	Next job to code (worth 20 points).
8	180 turns are timed, not measured.	Turns can be too far/short on different floors or battery levels.	Tune <code>TURN_180_S</code> . Later: use the IMU gyro.
9	LINE_VALUE does not match the hand-set camera angle.	After an obstacle check or the endzone, the camera may end up at the wrong angle.	Find the right number with <code>aim_camera.py</code> and put it in <code>LINE_VALUE</code> .
10	Pi folder permissions.	<code>scp</code> says 'Permission denied' for some files.	See section 7.
11	The Pi may have old copies of files.	Changes 'don't work' because they never reached the robot.	Always check the [version ...] line when the program starts.

7. When you see an error

You see...	It means...	Fix
Permission denied (when using scp)	You are not allowed to write into that folder on the Pi.	On the Pi run: <code>sudo chown -R nimesh:nimesh ~/rescue_robot</code> then try scp again. If it still fails: copy to the home folder (<code>scp FILE nimesh@rescue.local:~/</code>) then on the Pi: <code>sudo cp ~/FILE ~/rescue_robot/line_follow/</code>
ModuleNotFoundError: No module named 'something'	A file the program needs is missing on the Pi.	Copy that file (something.py) from the laptop with scp.
AttributeError: ... has no attribute ...	Two files on the Pi are different versions.	Copy ALL the main files again (Step 5, the long command).
Could not resolve hostname rescue.local	The laptop can't find the Pi.	Is the robot on? Same Wi-Fi? Wait a minute and try again.
Password asked again and again / wrong	Typed it wrong (letters are hidden).	Type slowly. Ask the team for the password.
Robot runs old behaviour	The Pi has an old copy.	Check the [version ...] line. Copy the files again.
PWMSoftwareFallback warning	Harmless. The servo uses software timing.	Ignore it.
Robot won't stop		Ctrl + C in the window. If that fails: pick it up and switch it off.

Team rules

1. **One change at a time.** Write down what you changed and what happened.
2. **Get the latest from the Pi first**, edit on your laptop, then copy back. Say in the team chat before you copy.
3. **Check the [version ...] line** every time you run.
4. **Do not use sudo** to run the scripts or copy files unless this guide says so (it causes the permission problem).
5. **Keep a hand near the robot** - be ready to press Ctrl + C or pick it up.
6. **Do not delete .bak files** - they are our undo button.
7. **Only one person pushes files to the Pi at a time.**

8. How to ask AI for help

You can use **any AI chat** (ChatGPT, Claude, Gemini, Copilot...). The AI does not know our robot, so the most important thing is to **give it the right information**. Follow these steps.

Step 1 - Start a NEW chat and paste this first

Copy this whole box into the chat as your first message (fill in the [brackets]):

```
I'm on a school RoboCup Junior Rescue Line team (Open division).
Robot: Raspberry Pi 5, Python 3, 2 front wheel BLDC motors
(set_speeds(left, right), -100..100), 8-sensor digital IR line array
(Parallax 28034, black=1), Pi camera on a servo (image rotated 180 deg
in camera_watcher.py), ToF distance sensor, rear claw with 2 servos.
Main program: line_follow/pid_line_follow_no_endzone.py
I'll paste the file and our team guide. Please:
- explain things simply, I'm not an expert
- suggest the SMALLEST change that fixes the problem
- tell me exactly which line / setting to change and to what
- don't rewrite the whole file unless I ask
My problem: [describe it]
```

Step 2 - Give it the files

Attach (or copy-paste) the **whole** file you are working on, e.g. `pid_line_follow_no_endzone.py`. If the problem is about the camera, also attach `camera_watcher.py`; for the zone, `endzone.py` and `zone_vision.py`. You can also attach **this PDF** so it knows how our robot works.

TIP: Get the file from YOUR laptop's `rescue_robot` folder - but get the latest from the Pi first (Step 3), so the AI sees the code that is actually on the robot.

Step 3 - Describe the problem like a detective

Tell the AI...	Example
What you did	Ran <code>pid_line_follow_no_endzone.py</code> on the U-turn tile.
What you expected	It should follow the line round the U-turn.
What actually happened	It drove off the top of the U and spun right forever.
What it printed (copy-paste ALL of it)	LINE LOST -> last turn was RIGHT -> spinning RIGHT ...
Any error message (copy-paste ALL of it)	Traceback (most recent call last): ... AttributeError: ...
Settings you changed recently	Kp from 50 to 15, LOST_TURN_SPEED 25.
A photo or video, if you can	Photo of the tile, or a <code>cam_check.jpg</code> picture.

Step 4 - Before you use the AI's answer, make a backup

Windows (on your laptop, in `line_follow`):

```
copy pid_line_follow_no_endzone.py pid_line_follow_no_endzone.py.bak_mychange
```

Mac (on your laptop, in `line_follow`):

```
cp pid_line_follow_no_endzone.py pid_line_follow_no_endzone.py.bak_mychange
```

Step 5 - Make the change, check it, test it

1. Make the change in VS Code and save.
2. Copy it to the Pi (Step 5 in section 2).
3. On the Pi, check there are no typing mistakes in the code:

On the Pi:

```
python3 -m py_compile line_follow/pid_line_follow_no_endzone.py
```

No message = OK. If it shows an error, paste that error back to the AI.

4. Run the robot and test.

Step 6 - Tell the AI what happened

Say "it worked" or "it's still doing X, here is what it printed: ...". Keep going in the **same chat** for the same problem, so it remembers. Start a **new chat** for a new problem.

Step 7 - If it got worse, undo

Windows (on your laptop):

```
copy pid_line_follow_no_endzone.py.bak_mychange pid_line_follow_no_endzone.py
```

(Mac: use cp instead of copy.) Then copy it to the Pi again.

AI rules

1. **Never paste the Pi password** (or any password) into an AI.
2. **The AI can be wrong.** It has never seen our robot. Always test, and check the robot still does the things that worked before.
3. **Ask it to explain** the change in simple words. If you don't understand it, ask again.
4. **One change at a time**, and write down what you changed.
5. **Tell the team** what you changed and push it to the Pi (one person at a time).